

Submit your answers in a PDF named pa01.pdf.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct if it has the same optimal cost.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scrabble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.

Steps:

```
cost = 0

if self.cost == "steps":

    for i in range(len(node.state)):

        if node.state[i] != self.default_goal[i]:

            cost += 1

return cost
```

Scrabble:

```
elif self.cost == "scrabble":

    for i in range(len(node.state)):

        if node.state[i] != self.default_goal[i]:

            letter = self.default_goal[i]

            //Check if the letter is in a specific letter

            //If so, have the cost be increment by the cost of that
letter

return cost
```

Frequency:

```
elif self.cost == "frequency":

    for i in range(len(node.state)):

        if node.state[i] != self.default_goal[i]:

            cost += 1

return cost
```

2. Explain what unavoidable remaining cost is lower-bounds.

For all of the cost methods, it would be noted that the minimum cost must be 1. For example, if you're using "Steps" with the initial state being dog and the goal state being dot, you would only need to change the 3rd index in order for the initial state to reach the goal state and this is the same with "Scrabble" with the letter with "t" and thus, the cost is just 1. As long as a change is needed, the total cost would always have a minimum of 1.

For frequency, the minimum cost would be 1 because no matter what action is taken for the algorithm to move to the next step, the cost required for it would be 1 + the cost of the word. This would mean that no matter how little the cost of the word can be, the minimum cost to travel would still be 1.

3. Explain why it cannot overestimate the true optimal remaining cost.

For steps, it can't overestimate the true optimal remaining cost because every cost always is 1, no matter what. This would mean that the amount of letters in the initial state that is different to the goal state would always equal the true optimal remaining cost. This would mean that the heuristic of the word pair would always equal the actual cost of the word pair and thus, can't be overestimated.

For scrabble, it contains the same reasoning for steps in that, the cost needed for a letter to turn into another letter of the goal state would be the same for both the heuristic and the actual cost since the cost is according to the table and since each of the indexes only have the letter be changed once, the cost needed for that index to change is only added in once.

For frequency, my heuristic makes it so that for every letter that's different from the goal state, it would add 1 to the total estimated cost. This would make it so that it can't overestimate the actual cost because actual cost would have it so that every change would 1 + the rarity cost needed for it to turn into a specific word. It's due to the rarity cost added into the actual cost that it would always be higher than the heuristic cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic "is a lower bound" is insufficient.

Question 2: Does the heuristic actually help? [10 points]

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	768	5	16	97.9167%
WHY → ASK	Scrabble	9	750	9	13	98.267%

WHY → ASK	Frequency	38.855406 52	803	38.855406 52	752	6.351%
BEAR → DUCK	Steps	4	682	4	6	99.12%
BEAR → DUCK	Scrabble	11	2977	11	6	99.7985%
BEAR → DUCK	Frequency	38.240228 4	1075	38.240228 4	715	33.488%

Calculate the percentage reduction as:

$$100 \times ((\text{UCS expansions} - \text{A}^* \text{ expansions}) / \text{UCS expansions})$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Both of the USC and A* ended up returning the same cost.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

The heuristic for Scrabble would end up creating the highest reduction percentage for both word pairs.

3. For each word pair, under which cost measure was the heuristic least informative?

For each word pair, the frequency can be considered to be the least informative since they produced the least reduction percentage.

4. Explain the differences using the definitions of your heuristics, not runtime alone.

According to the heuristics for “Steps”, it would be shown that both USC and A* would’ve had the same total cost since changing a letter would only require a cost of 1. The same could be applied for “Scrabble” since the cost to change a

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scrabble**, and **frequency**—determine whether the optimal

cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

1. State **yes** or **no**.

Steps: Yes

Scrabble: No

Frequency: No

2. Justify your conclusion.

For steps, the cost needed to move to the next state is 1 and that the only change would be changing the letter of the index to a letter from the goal state. If the action was reversible, that would mean the letter would change back to its original state which would only have a cost of 1, as well.

For scrabble, it wouldn't be the same because the cost would depend on the letter that an index was replaced with and not every letter would have the same cost. Not only that but if a letter from the initial state was changed to a letter from the goal state, it might have a cost that's different than if it was a letter from the goal state that was changed to the original state.

For frequency, the same argument can be applied from scrabble, in which the cost to go from W_1 to W_2 would not be the same as from W_2 to W_1 because each word that it turns into has a rarity cost and that the rarity cost for each word is different.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

For scrabble, an example can be given where the initial state is dog and the next state is cog. In order for d to turn into c, the cost needed for it would be 3 but if the action was reversible, then for c to turn into d, the cost would have to be 2. This would prove that the cost from W_1 to W_2 is not the same as from W_2 to W_1 even though it can be shown that the action is reversible.

For frequency, an example can be given where the initial state is dog and the next state is cog. For the state, which is dog, to turn into cog, the rarity cost for it is 11.393166 but if the action was reversible, then for dog to turn into cog, the cost would have to be 5.819885. This would prove that the cost from W_1 to W_2 is not the same as from W_2 to W_1 even though it can be shown that the action is

reversible.

Question 4: LLM heuristic audit [10 points]

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

I'm using the free/basic version of Bing Copilot and the current model version is 2.20260921.60.0.

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.



The proposed heuristic that the ai suggested was a rarity-aware heuristic.

2. A precise formula or pseudocode for the proposed heuristic.

Proposed rarity-aware heuristic

Let

$$r_{\min} = \min_{w \in \text{dictionary}} \text{rarity}(w).$$

Because rarity values are nonnegative, $r_{\min} \geq 0$.

Define

$$h_R(n) = h_0(n) (1 + r_{\min}).$$

Equivalently,

$$h_R(n) = \text{HammingDistance}(n, \text{goal}) \times \left(1 + \min_w \text{rarity}(w)\right).$$

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

It can be considered as admissible because the cost in order to travel towards a specific state is $1 + \text{rarity}(w)$ and since the cost of rarity word would be greater than or equal to the minimum possible cost of a rarity word, this would mean that the cost to travel to a state would always be at least $1 +$ the minimum rarity word. Since, the heuristic would be around

the actual cost and the actual cost would be greater than the minimum estimated cost, this would prove that the heuristic estimated cost could never overestimate the actual cost of a word pair.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

I view the proposal as valid and thus, wouldn't require a repair nor a safe alternative.

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Assuming that I prompt this in the model, it was shown that the using the A* algorithm, the proposed heuristic would have a similar solution cost to the original heuristic and that the amount of nodes expanded would be at most the same amount as the original heuristic. Not only that but it stated even though it can be considered to be more admissible, their similarities makes it so that they can be considered to be the same due to the proposed heuristic rarity value being extremely small with it being 0.016226212. As a result, its reduction isn't considered to be big enough to be considered as an improvement and thus, shows that both of the heuristics are similar if not, the same.

.